المحاضرة 1:
مقدمة في قواعد البيانات الموزعة

أ. سعاد محمد عثمان

المحتويات

- مقدمة وخلفية نظرية.
- أنظمة قواعد البيانات الموزعة (Distributed Database Systems).
- مزايا وعيوب أنظمة قواعد البيانات الموزعة.

الخلفية

- Database: مجموعة من البيانات توفر الاستمرارية والتكامل.
- DBMS: برنامج يمكن المستخدمين النهائيين أو مبرمجي التطبيقات من مشاركة البيانات وإدارتها، ويوفر طريقة منهجية لإنشاء البيانات وتحديثها واسترجاعها وتخزينها في قاعدة البيانات.

قاعدة البيانات الموزعة (Distributed Database)

- A distributed database (DDB): جموعة من قواعد البيانات المتعددة المترابطة منطقيا والموزعة عبر شبكة حاسوبية.
- A distributed database management system (D-DBMS): رنامج الذي يدير الـ DDB ويوفر آلية وصول تجعل هذا التوزيع شفافا للمستخدمين.
- Distributed database system (DDBS) = DDB + D-DBMS

خصائص قواعد البيانات الموزعة (DDB)

- قواعد البيانات في المجموعة مترابطة منطقيا مع بعضها البعض، وغالبا تمثل قاعدة بيانات منطقية واحدة.
- البيانات مخزنة فيزيائيا عبر مواقع (sites) متعددة، ويمكن إدارة البيانات في كل موقع بواسطة DBMS مستقل عن باقي المواقع.
- المعالجات في المواقع متصلة عبر شبكة، وليس لديها أي تهيئة متعددة المعالجات (multiprocessor configuration).

العوامل المشجعة على الانتقال إلى DDBMS

العوامل التالية تشجع على الانتقال إلى DDBMS:

- الطبيعة الموزعة للوحدات التنظيمية (Distributed Nature of Organizational Units).
- الحاجة إلى مشاركة البيانات (Need for Sharing of Data).
- دعم OLTP و OLAP معا.
- استرداد قاعدة البيانات (Database Recovery).
- دعم تطبيقات برمجية متعددة (Multiple Application Software).
- إمكانية العمل المنفصل (Disconnected operation possible).

المحاضرة 2:
قاعدة البيانات الموزعة

أ. سعاد محمد عثمان

المحتويات

- مفاهيم أنظمة قواعد البيانات الموزعة.
- معالجة البيانات الموزعة (Distributed Data Processing).
- طبقات الشفافية (Layers of Transparency).

مفاهيم أنظمة قواعد البيانات الموزعة

- مجموعة من البيانات المشتركة المترابطة منطقيا.
- البيانات مقسمة إلى أجزاء (fragments).
- قد تكون الأجزاء مكررة (replicated).
- المواقع (sites) مرتبطة بشبكة اتصالات.
- البيانات في كل موقع تحت سيطرة DBMS.

معالجة البيانات الموزعة (Distributed Data Processing)

- Distributed Processing مقابل Distributed Databases

المصطلحان مترابطان لكنهما يختلفان في المعنى:

- Distributed database: مجموعة من قواعد البيانات في نظام موزع يمكن أن تظهر للتطبيقات كمصدر بيانات واحد.

- Distributed processing: العمليات التي تحدث عندما يوزع تطبيق ما مهامه على حواسيب مختلفة في الشبكة. مثلا، تطبيق قاعدة البيانات يوزع مهام العرض على العملاء ويترك لخادم قاعدة البيانات الخلفي إدارة الوصول المشترك.

- إدارة شفافة للبيانات الموزعة والمكررة.
- موثوق للبيانات عبر المعاملات الموزعة (distributed transactions).
- تحسين الأداء (Improved performance).
- سهولة توسعة النظام (Easier system expansion).

مزايا قواعد البيانات الموزعة
(Promises of Distributed Databases)

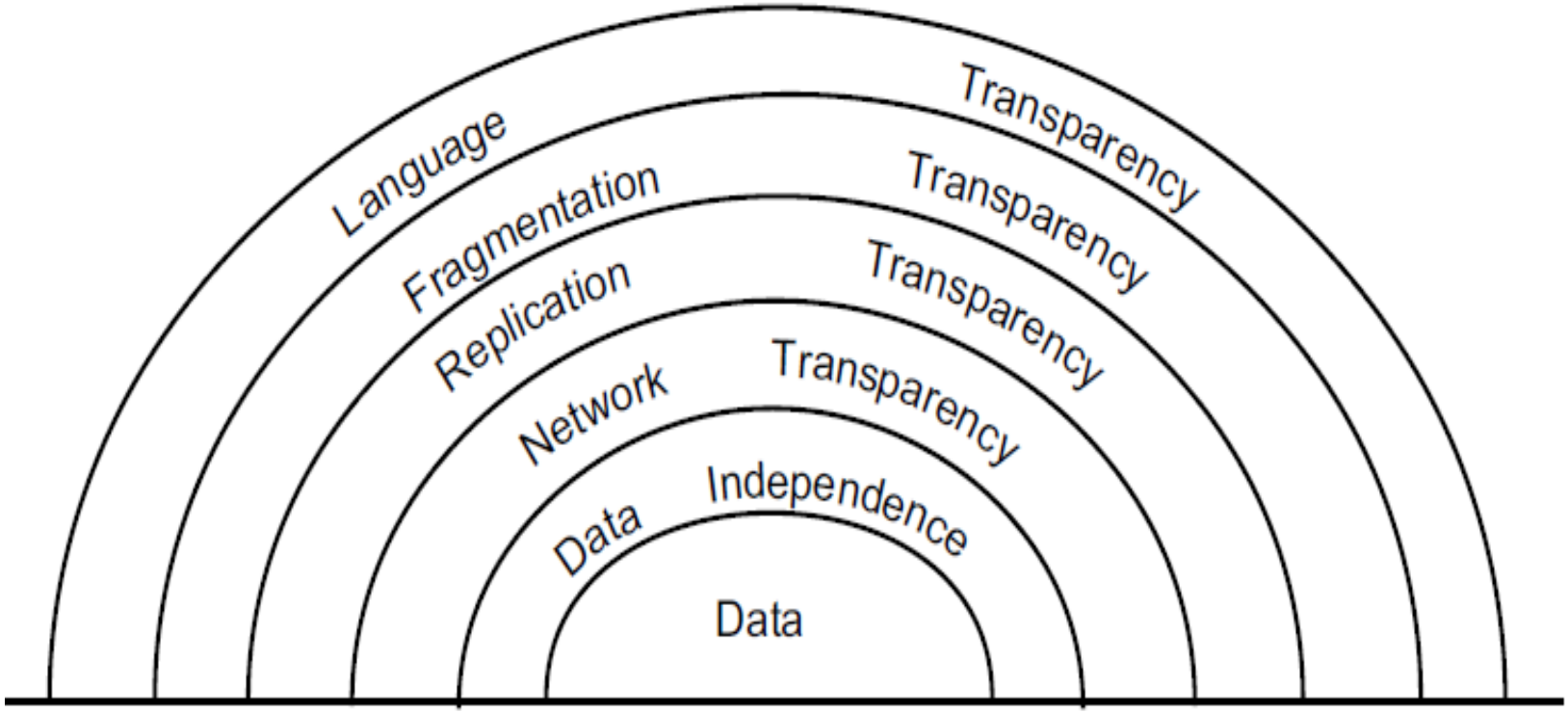
ام الشفاف (Transparent system): يخفي تفاصيل التنفيذ عن المستخدمين.

- ميزة الـ DBMS الشفاف الكامل هي المستوى العالي من الدعم الذي يوفره لتطوير التطبيقات المعقدة.

الإدارة الشفافة للبيانات الموزعة والمكررة
(Transparent Management of Distributed and Replicat

طبقات الشفافية (Layers of Transparency)

- يحتاج النظام إلى التعامل مع عدد من أنواع الشفافية المختلفة.



استقلالية البيانات (Data Independence)

- تشير إلى مناعة تطبيقات المستخدم ضد التغييرات في تعريف البيانات وتنظيمها، والعكس صحيح.

- أنواع استقلالية البيانات:

- الاستقلالية المنطقية للبيانات (Logical data independence)
- الاستقلالية الفيزيائية للبيانات (Physical data independence)

استقلالية البيانات (Data Independence)

- **Logical data independence:** مناعة تطبيقات المستخدم ضد التغييرات في البنية المنطقية (المخطط - schema) لقاعدة البيانات.

- **Physical data independence:** تعنى بإخفاء تفاصيل هيكل التخزين عن تطبيقات المستخدم.

شفافية الشبكة (Network Transparency)

- يشار إلى هذا النوع بشفافية الشبكة (network transparency) أو شفافية التوزيع (distribution transparency).
- يجب حماية المستخدم من التفاصيل التشغيلية للشبكة، وربما إخفاء وجودها كليا.
- تتطلب شفافية التوزيع أن لا يحتاج المستخدم لتحديد مكان البيانات.
- نوعا شفافية التوزيع:
  - شفافية الموقع (location transparency)
  - شفافية التسمية (naming transparency)

شفافية الشبكة (Network Transparency)

- **Location transparency:** الأمر المستخدم لتنفيذ مهمة ما مستقل عن موقع البيانات والنظام الذي تنفذ عليه العملية.
- **Naming transparency:** يعني أن لكل كائن في قاعدة البيانات اسما فريدا.

شفافية التكرار (Replication Transparency)

- مسألة الشفافية هنا هي: هل يجب أن يعلم المستخدمون بوجود نسخ متعددة؟ أم يجب أن يتعامل النظام مع إدارة النسخ ويتصرف المستخدم كما لو كانت هناك نسخة واحدة فقط من البيانات؟
- تذكر: شفافية التكرار (replication transparency) تشير فقط إلى وجود النسخ، وليس إلى مواقعها الفعلية.

شفافية التجزئة (Fragmentation Transparency)

- عند تجزئة كائنات قاعدة البيانات، يجب التعامل مع استعلامات المستخدم التي تطبق على علاقات كاملة لكنها تنفذ على علاقات جزئية (subrelations). أي إيجاد استراتيجية معالجة الاستعلامات بناء على الأجزاء لا العلاقات.

- يتطلب هذا عادة ترجمة من الاستعلام العالمي (global query) إلى عدة استعلامات جزئية (fragment queries).

من يوفر الشفافية؟

- يمكن تحديد ثلاث طبقات مستقلة يمكن فيها تقديم خدمات الشفافية:

- طبقة الوصول (access layer).
- مستوى نظام التشغيل (operating system level).
- الطبقة الثالثة: داخل DBMS نفسه.

Question?

المحاضرة 5:
تصميم قواعد البيانات الموزعة
(Distributed Database Design)

أ. سعاد محمد عثمان

هناك استراتيجيتان رئيسيتان لتصميم قواعد البيانات الموزعة:

- أسلوب القمة للقاعدة (Top-down approach)
- أسلوب القاعدة للقمة (Bottom-up approach)

تصميم قواعد البيانات الموزعة
(Designing Distributed Databases)

أسلوب القاعدة للقمة (Bottom-up approach)

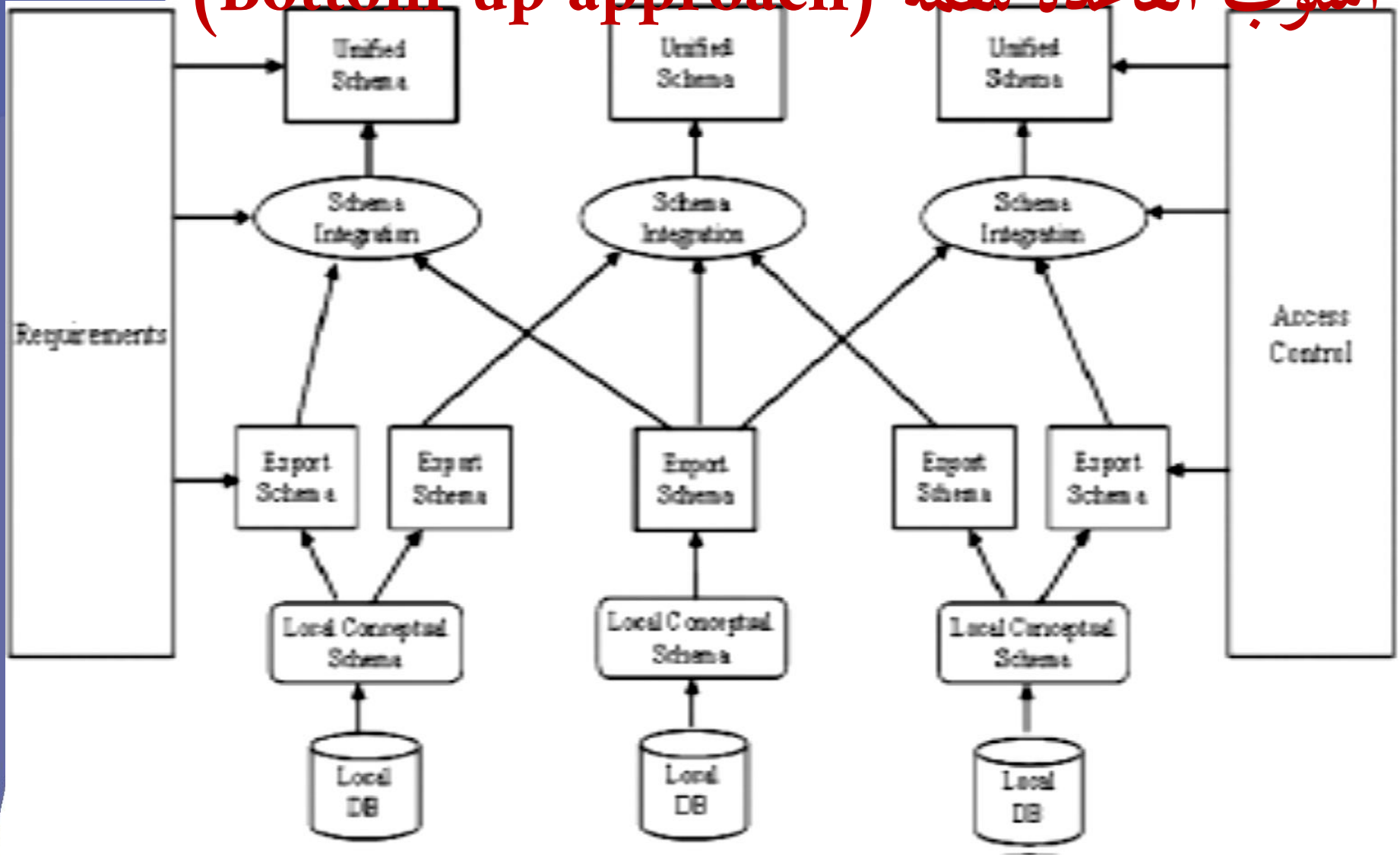
- يعتمد على البيانات الموزعة الموجودة مسبقا داخل المنظمة، ويستخدم عملية تسمى تكامل المخطط (schema integration) لإنشاء مخطط موحد واحد على الأقل.

- Schema integration: عملية تستخدم مجموعة من عناصر النموذج المفاهيمي الموجودة، المصدرة من نموذج LCM واحد أو أكثر، لإنشاء نموذج متكامل دلاليا (مخطط موحد واحد).

أسلوب القاعدة للقمة (Bottom-up approach) - تابع

- **Unified Schema:** مخطط متكامل وغير مكرر ومتسق لمجموعة من أنظمة قواعد البيانات المحلية.

أسلوب القاعدة للقمة (Bottom-up approach)



نشاط تصميم التوزيع (Distribution Design Activity)

- هدف أي توزيع للبيانات هو توفير توافرية أعلى وموثوقية أكبر وتحسين وقت الوصول للاستعلامات.

- أنشطة تصميم التوزيع:
  - التخصيص Allocation (غير مكرر، غير مجزأ)
  - التجزئة Fragmentation
  - التكرار Replication
  - المختلط Mixed

التخصيص (Allocation)

- يسمح للمصمم بوضع جداول مختلفة من قاعدة بيانات معينة في مواقع مختلفة.
- الفكرة: ينبغي وضع البيانات قريبة من الموقع الذي تحتاج فيه أكثر.
- فائدة هذا الأسلوب: تقليل مكون الاتصالات في تكلفة المعالجة.

التجزئة (Fragmentation)

- أسلوب التجزئة يقسم الجدول إلى قطعتين أو أكثر تسمى fragments أو partitions، ويسمح بتخزين هذه القطع في مواقع مختلفة.

- التجزئة توفر زيادة في التوازي (parallelism)، والوصول (access)، والتعافي من الكوارث (disaster recovery)، والأمان والخصوصية (security/privacy).

التجزئة (Fragmentation)

- هناك ثلاثة بدائل للتجزئة:

 التجزئة الرأسية (Vertical fragmentation)

 التجزئة الأفقية (Horizontal fragmentation)

 التجزئة الهجينة (Hybrid fragmentation)

- يمكن تصنيف التجزئة الأفقية إلى نوعين:

 التجزئة الأفقية الأولية (PHF - Primary Horizontal Fragmentation)

 التجزئة الأفقية المشتقة (DHF - Derived Horizontal Fragmentation)

التجزئة (Fragmentation)

- عند التفكير في التجزئة، يحتاج المصممون لتحديد درجة الحبيبية (granularity) لكل جزء. بمعنى آخر:
  كم من أعمدة و/أو صفوف الجدول ينبغي أن يكون في الجزء؟

التجزئة الرأسية (Vertical Fragmentation)

- التجزئة الرأسية (VF): تجمع أعمدة الجدول في أجزاء.
- يجب إجراء التجزئة الرأسية بحيث يمكن إعادة بناء الجدول الأصلي من الأجزاء.
- هذا الشرط يسمى 'قابلية إعادة البناء' (reconstructiveness)، ويستخدم لإعادة بناء الجدول الأصلي عند الحاجة.

مثال (Example)

EmpID	Name	Loc	Sal	DOB	Dept
283948	Joe	LA	25,000	2/6/43	Maintenance
109288	Larry	New York	35,200	12/3/52	Payroll
284003	Moe	LA	43,000	7/12/56	Maintenance
320021	Sam	New York	53,500	8/30/47	Production
123456	Steve	Minneapolis	67,000	5/14/78	Management
334456	Jack	New York	55,000	5/30/67	Production
222222	Saeed	Minneapolis	34,000	4/27/59	Management

EMP Table

مثال على التجزئة الرأسية (VF Example)

- مثال:

```
Create table EMP_SAL as
  Select EmpID, Sal
  From EMP;

Create table EMP_NON_SAL as
  Select EmpID, Name, Loc, DOB, Dept
  From EMP;
```

شفافية التجزئة (Fragmentation Transparency)

- بعد التجزئة، لن يخزن جدول EMP فيزيائيا في أي مكان.
  لكن لتوفير شفافية التجزئة — أي عدم إلزام المستخدمين بمعرفة أن جدول EMP مجزأ — سنستخدم جملة JOIN في كل موضع يطلب فيه جدول EMP.

مثال

```
Select EMP_SAL.EmpID, Sal, Name, Loc, DOB, Dept
From EMP_SAL, EMP_NON_SAL
Where EMP_SAL.EmpID = EMP_NON_SAL.EmpID;
```

التجزئة الأفقية (Horizontal Fragment)

- التجزئة الأفقية (HF): تجمع صفوف الجدول بناء على قيم عمود أو أكثر.
- يجب أن يحتوي كل جزء أفقي على جميع أعمدة الجدول الأصلي.
- لإنشاء جزء أفقي من جدول، تستخدم جملة SELECT. مثلا:

Select * from R where C;

الأفقية الأولية (Primary Horizontal Fragmentation - PHF)

- PHF: تقسم الجدول أفقيا بناء على قيم عمود أو أكثر.
- مثال: جدول EMP. لنفترض وجود ثلاثة فروع، وكل موظف يعمل في فرع واحد فقط. لسهولة الاستخدام، نقرر تخزين معلومات الموظف في خادم DBMS بالفرع الذي يعمل فيه.

مثال

```
Create table MPLS_EMPS as Select *
  From EMP Where Loc = 'Minneapolis';

Create table LA_EMPS as Select *
  From EMP Where Loc = 'LA';

Create table NY_EMPS as Select *
  From EMP Where Loc = 'New York';
```

شفافية التجزئة الأفقية

- بعد التجزئة، لن يخزن جدول EMP فيزيائيا في أي مكان.
- لتوفير شفافية التجزئة الأفقية، يجب أن نكون قادرين على إعادة بناء جدول EMP من أجزائه الأفقية.
- لتحقيق ذلك، سنستخدم جملة UNION في كل موضع يطلب فيه جدول EMP.

مثال

```
Select * from MPLS_EMPS
Union
Select * from LA_EMPS
Union
Select * from NY_EMPS;
```

التجزئة الأفقية المشتقة

- DHF: تجزئ جدولا بناء على طريقة تجزئة جدول آخر.
  وهي أسلوب يجزئ صفوف جدول بناء على قيم أعمدة جدول آخر.

- تستخدم DHF عادة لجدولين يوحدان طبيعيا (وبكثرة).
  لذلك، تخزين الأجزاء المتناظرة من الجدولين في نفس الموقع
  يسرع عملية الدمج (JOIN) بين الجدولين.

مثال

```
Create table PROJ1 as
  Select Pno, Pname, Budget, PROJ.Dno
  From PROJ, MPLS_DEPTS
  Where PROJ.Dno = MPLS_DEPTS.Dno;
```

```
Create table PROJ2 as
  Select Pno, Pname, Budget, PROJ.Dno
  From PROJ, NY_DEPTS
  Where PROJ.Dno = NY_DEPTS.Dno;
```

```
Create table PROJ3 as
  Select Pno, Pname, Budget, PROJ.Dno
  From PROJ, LA_DEPTS
  Where PROJ.Dno = LA_DEPTS.Dno;
```

التجزئة الهجينة (Hybrid Fragmentation - HyF)

- HyF: تستخدم مزيجا من التجزئة الأفقية والرأسية.

- هناك أسلوبان:
  - الأول: إنشاء مجموعة من الأجزاء الأفقية ثم تجزئة بعضها رأسيا.
  - الثاني: إنشاء مجموعة من الأجزاء الرأسية ثم تجزئة بعضها أفقيا.

التجزئة الهجينة (Hybrid Fragmentation)

- يوفر هذا الأسلوب أقصى مرونة للمصممين،
  لكنه الأكثر تكلفة من حيث إعادة بناء الجدول الأصلي.

مثال

```
Create table NON_SAL_MPLS_EMPS as
  Select EmpID, Name, Loc, DOB, Dept
  From EMP Where Loc = 'Minneapolis';

Create table NON_SAL_LA_EMPS as
  Select EmpID, Name, Loc, DOB, Dept
  From EMP Where Loc = 'LA';

Create table NON_SAL_NY_EMPS as
  Select EmpID, Name, Loc, DOB, Dept
  From EMP Where Loc = 'New York';
```

التكرار (Replicated)

- كلما زاد عدد النسخ من جدول/جزء، كان الاستعلام عنه أسهل. لكن كلما زادت النسخ، كان تحديثها جميعا أكثر تعقيدا واستهلاكا للوقت.

- هناك بديلان للتكرار:

 التكرار الكامل (Fully replicated)

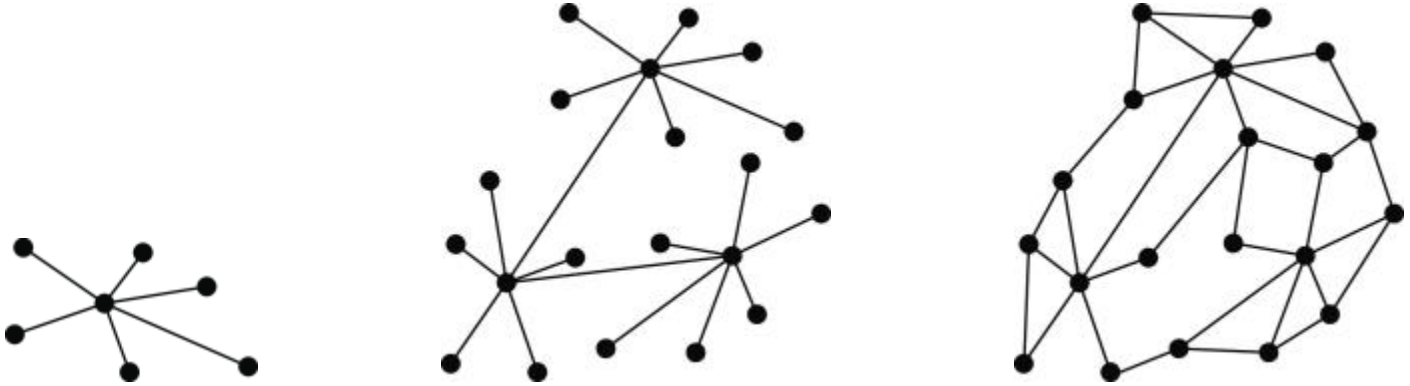
 التكرار الجزئي (Partially replicated)

التكرار (Replicated)

- Fully Replicated (التكرار الكامل): يخزن نسخة واحدة من كل جدول في كل موقع. يوفر أفضل أداء ممكن للاستعلامات.
- Partially Replicated (التكرار الجزئي): ينشئ المصمم نسخا من بعض الجداول (أو الأجزاء) ويخزنها في مواقع مختلفة، استنادا إلى اعتقاد بأن تكرار الوصول لجداول قاعدة البيانات ليس متساويا.

(Distributed Sys

دمة



(Decentra موزع (Distributed)

دمة

ظمة شبكية موجودة في

مقدمة

دة:

(Support sharing

(Distribu

(Resource Shari

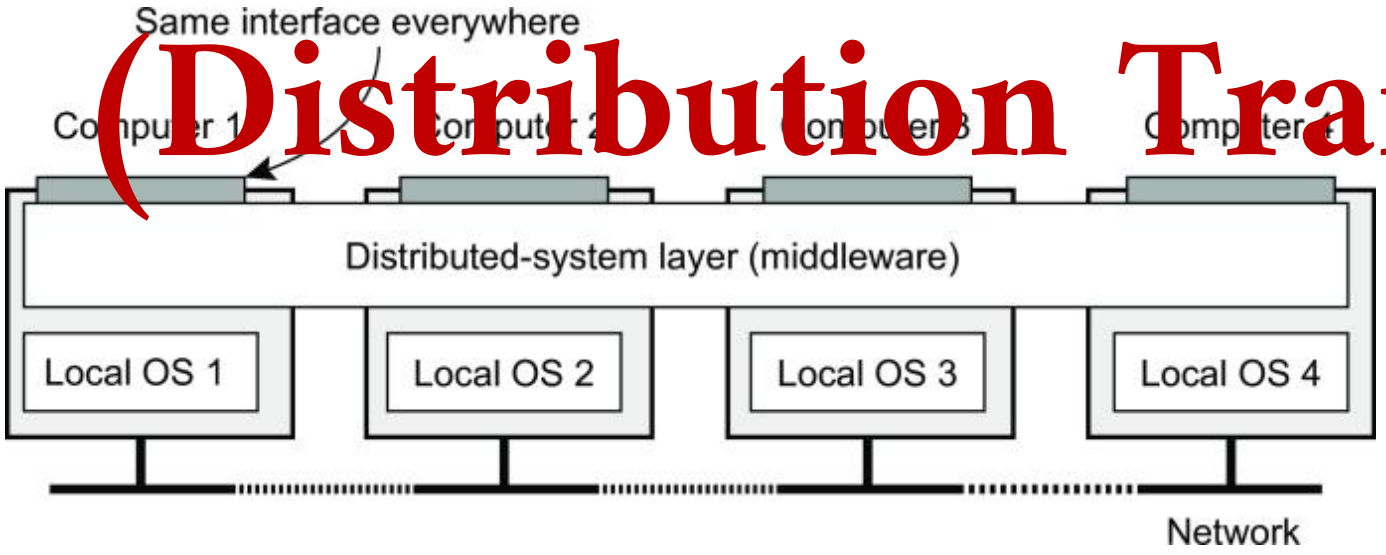
(Cloud-based)

Peer-t

(Shared m

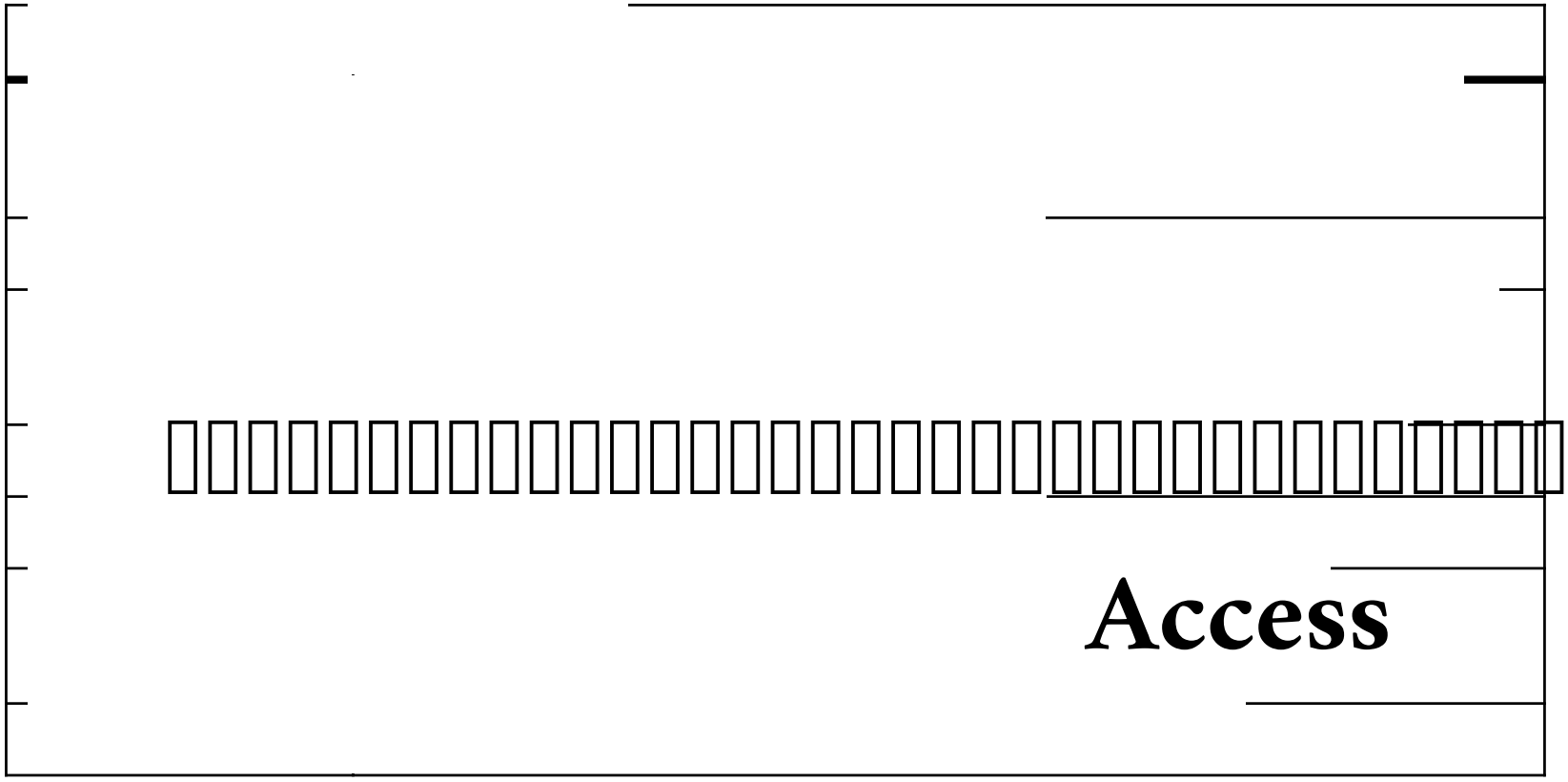
(Shared W

(Distribution Transpa



ء حقيقة أن

ب متعددة،



Relocation

Migrati

(Degree of Tran

مبالغا فيه:

كن إخفاؤها.

ى نظريا وعمليا.

وب معطل.

قبل انهياره.

(location-based services).

مختلفة.

الإبلاغ عن الخادم

بلة).

(Openn

:(Open Dis

لة أو دمجها

(Policies vs. Mechani

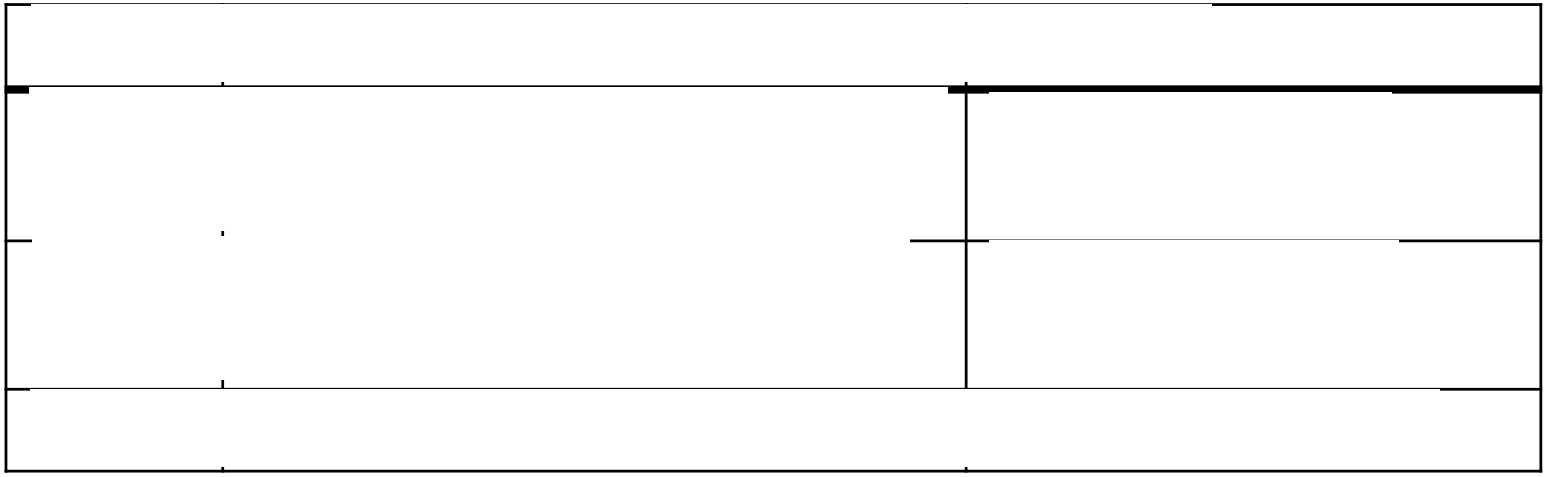
مخزنة مؤقتا؟

؟

لنطاق؟

خلال الفترة [t,0)

- MTTF (Mean Time T



شال

□□□

(Handling

مثال

Fault prevention ملين

Fault tolerance ن مستقلين

Fault removal المهملين

Fault forecasting

ة (Scalability)

دمون صفة 'scalable'

(Size

(Size Scalal

في الحلول المركزية:

ن الـ CPUs والأقراص

زية

WA: كثير من الأنظمة
متزامنة؛ زمن الاستجابة

لإدارة والأمان.

: مشاركة موارد باهظة

(Techniques f

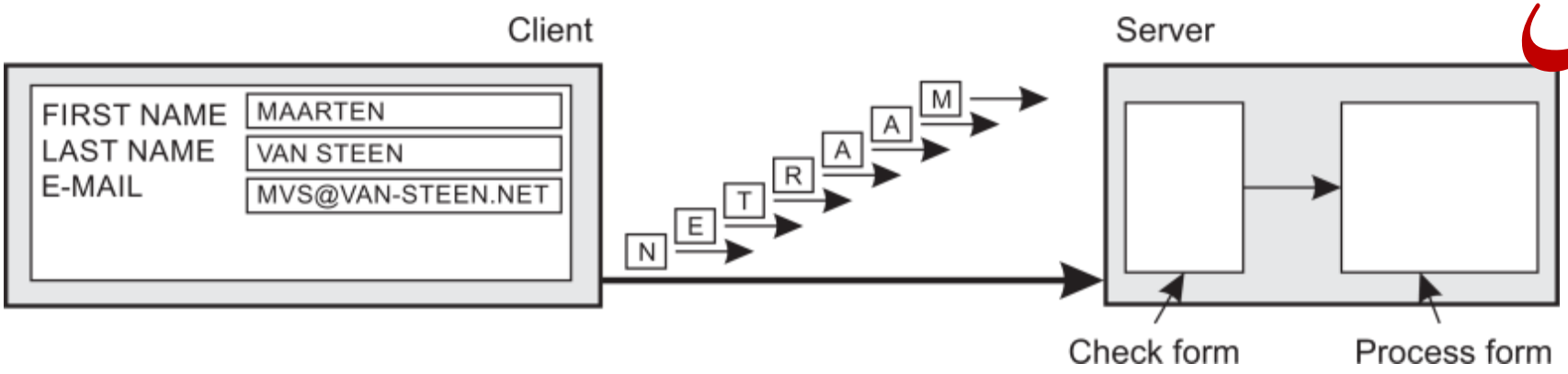
:(Hide communicati

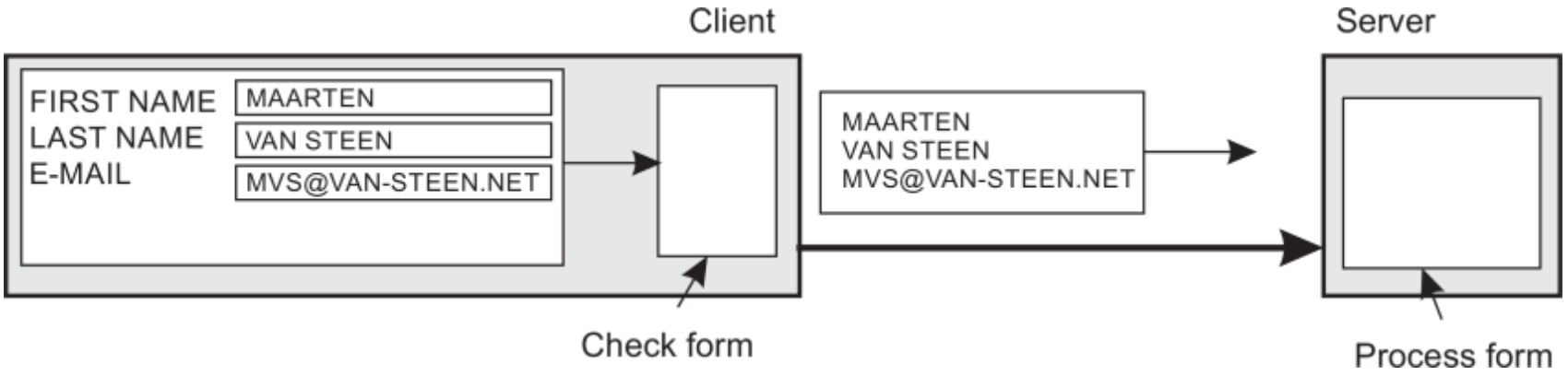
.(asynchronous communica

ردة.

النموذج.

ميل


Client
Server
FIRST NAME
LAST NAME
E-MAIL
MAARTEN
VAN STEEN
MVS@VAN-STEEN.NET
N
E
T
R
A
A
M
Check form
Process form


Client
Server
FIRST NAME
LAST NAME
E-MAIL
MAARTEN
VAN STEEN
MVS@VAN-STEEN.NET
MAARTEN
VAN STEEN
MVS@VAN-STEEN.NET
Check form
Process form

Ja وسكريبتات).

.

(Replication and Cach

ختلفة:

رة

(Mirrore

المتصفحات والبروكسي)

(The Problem with I

) يؤدي للتعارض:

بة.

نا عاما

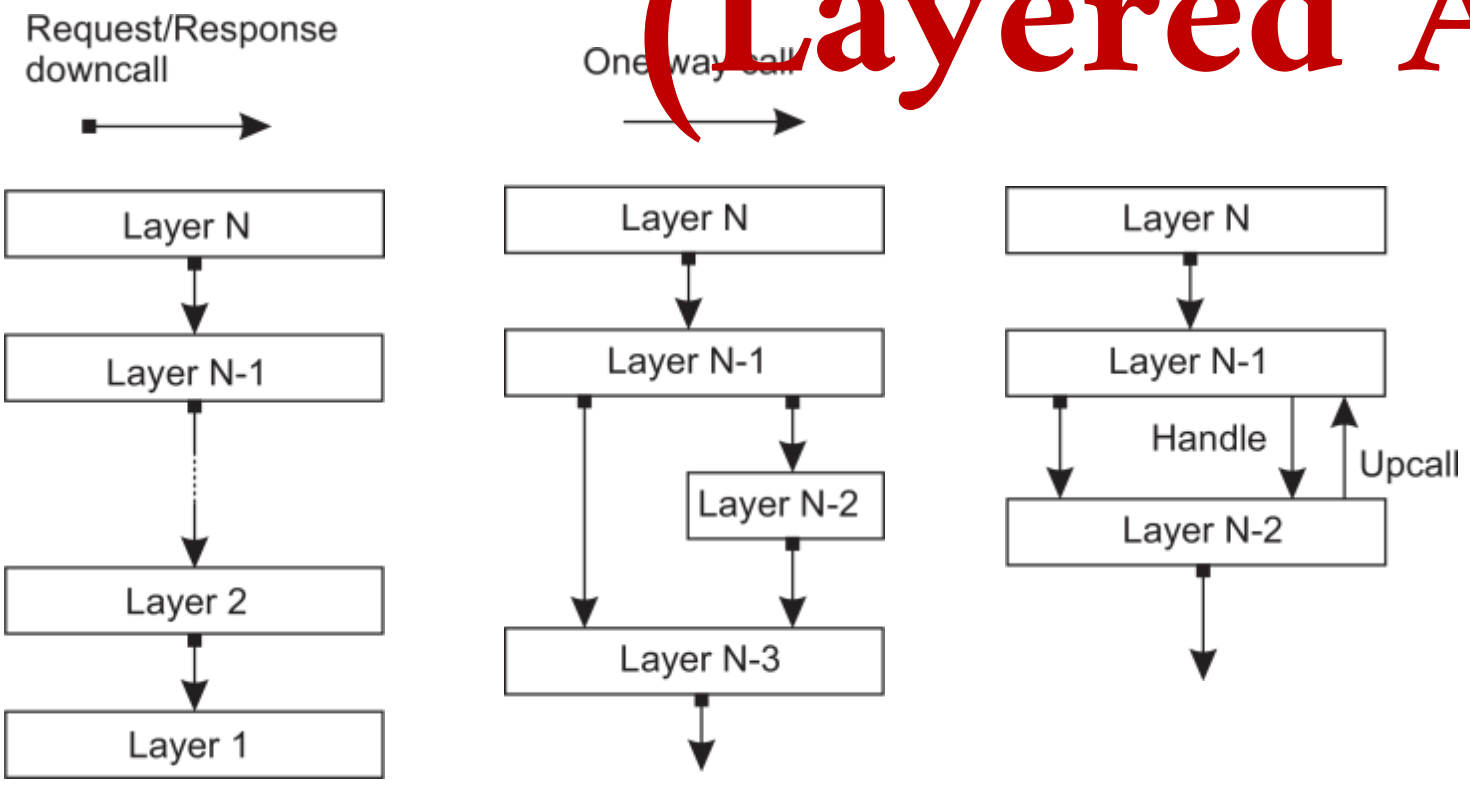
(g

(Distributed Sys
(Architectures)

(Architectu

ل ذات واجهات محددة

(Layered Arch



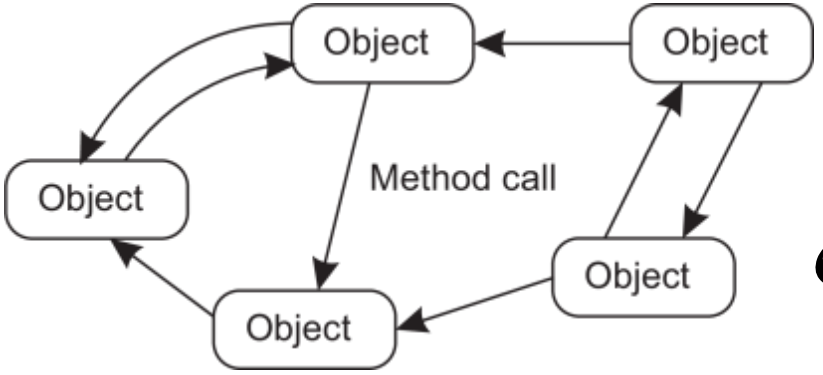
(Application

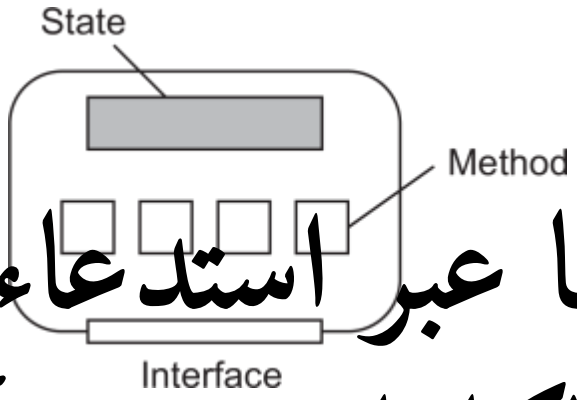
(Application): تحتوي وحدات
خارجية.
حتوي وظائف التطبيق

(Application

(Application

(Object-b



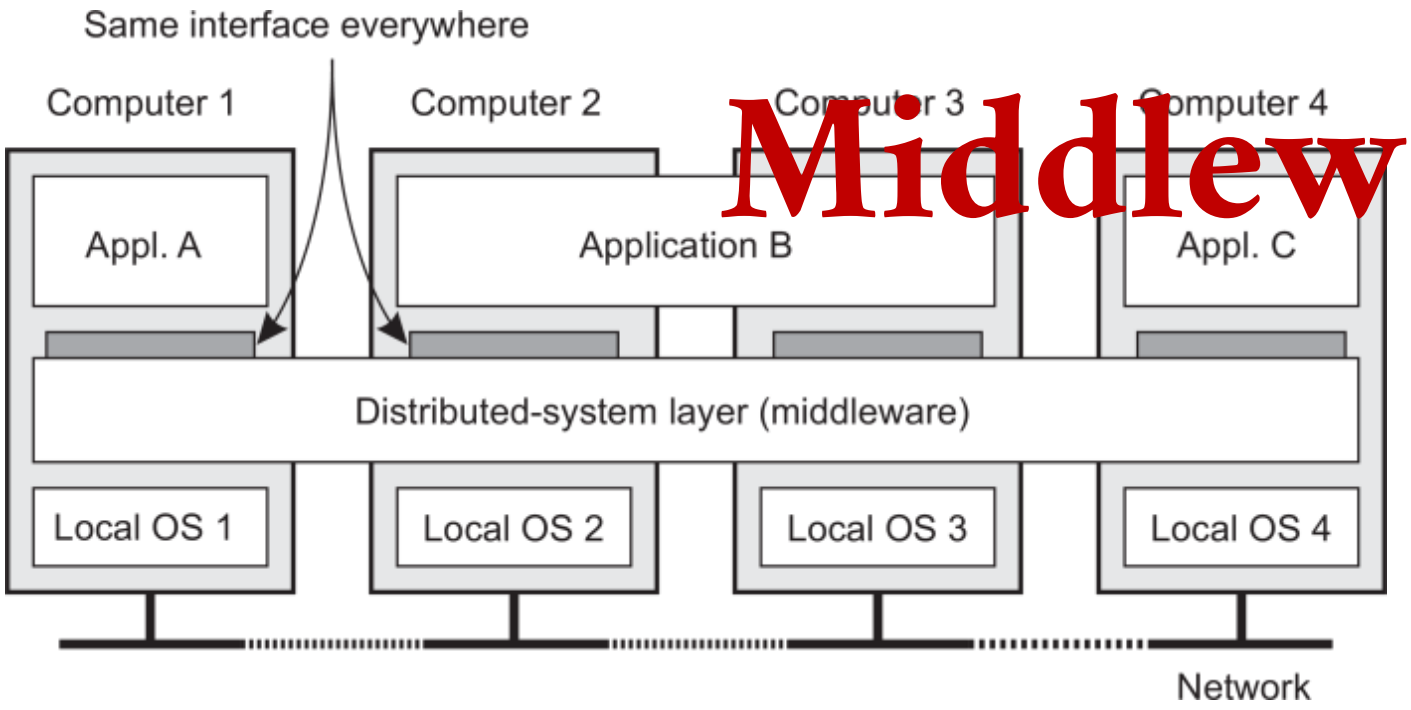


ببعضها عبر استدعاءات
وضع الكائنات على أجهزة
بكة.

ها مكونات

عها وتعديلها

Middleware



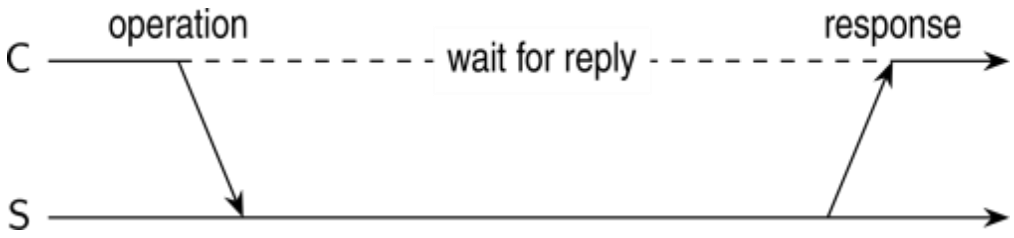
Mid

تحتاج التطبيقات لتنفيذها

Middleware

(legacy components

Clien

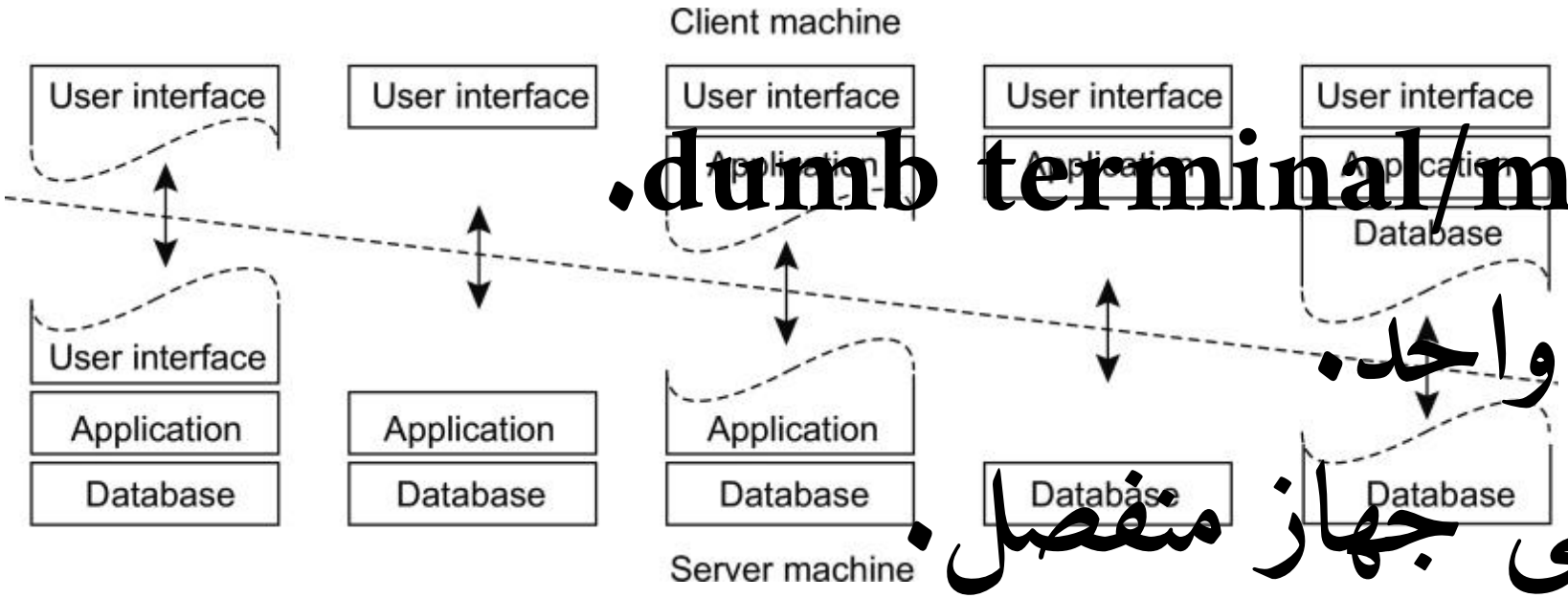


.(se

.(clien

أجهزة مختلفة.

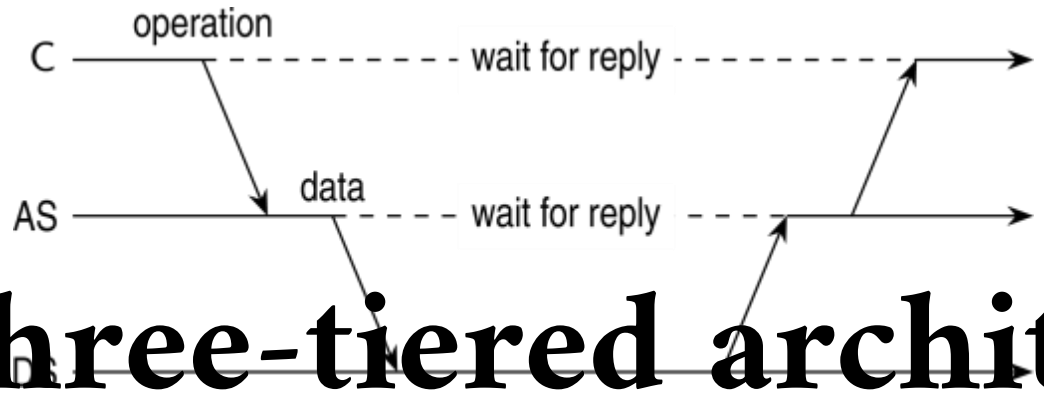
(request/ في استقبال الرسالة



.dumb terminal/mainf

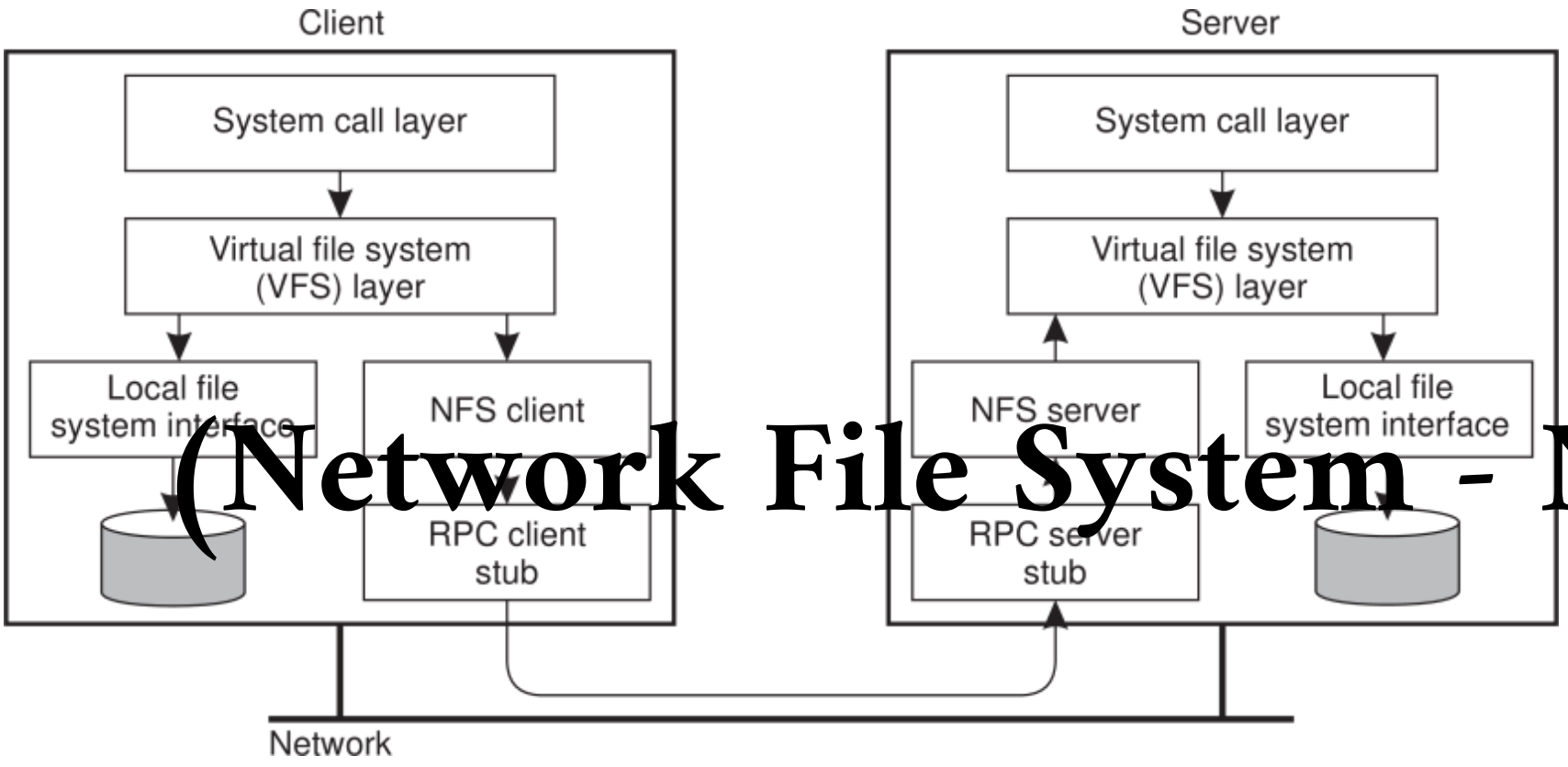
clie واحد.

ة على جهاز منفصل.

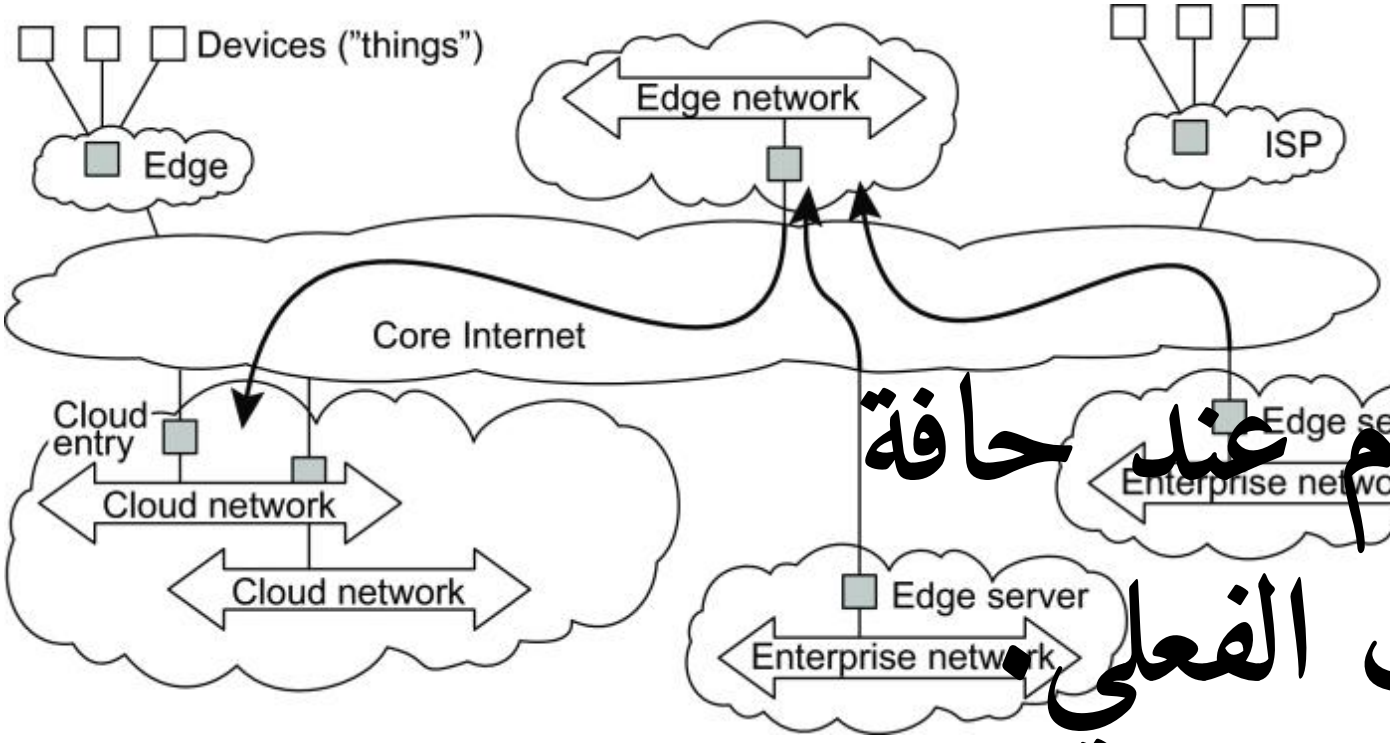


(Three-tiered architecture)

(Network File System - NF



(Edge-server Architec



الخوادم عند حافة
الإنترنت الفعلي.

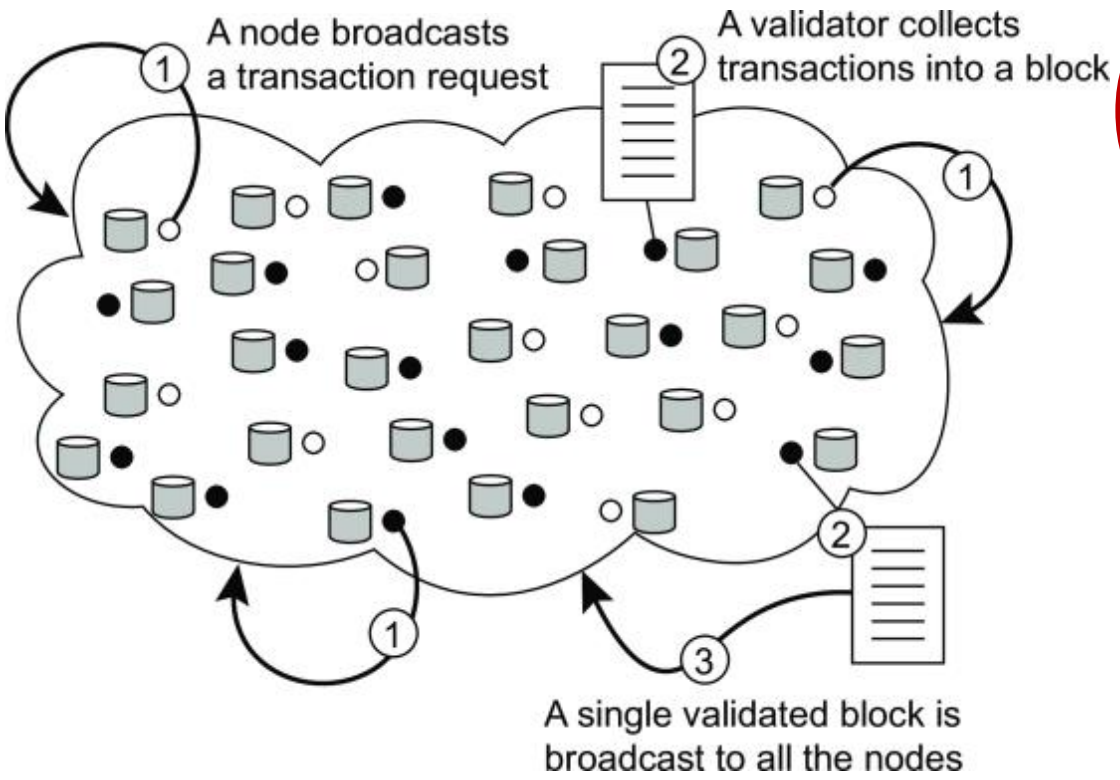
(Edge Infrastructure

(Latency and : مهم

ضي. كثيرون

للسحابة.

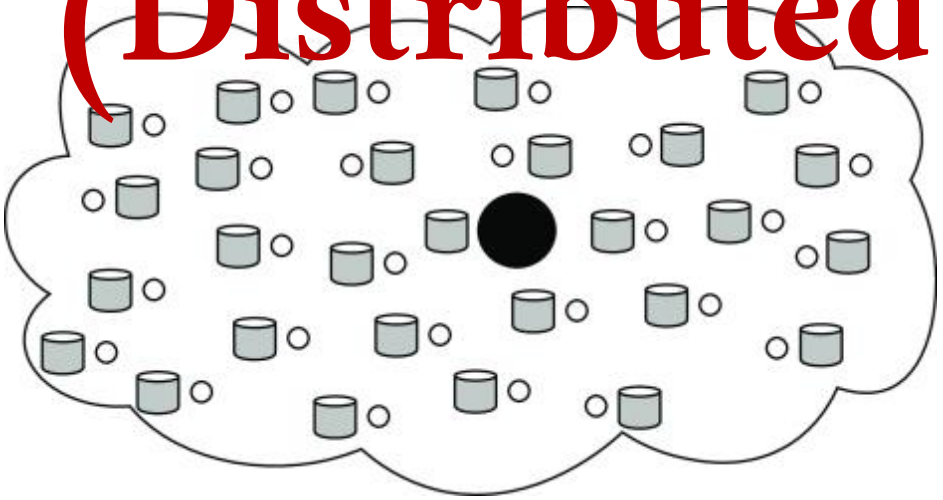
ن الاتصال بالسحابة



(Bloc

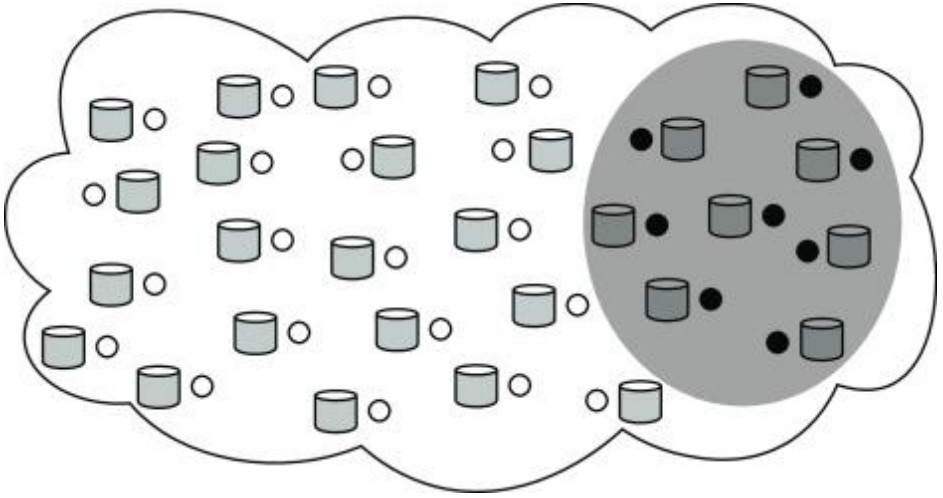
وتضاف فقط

(Distributed Consen



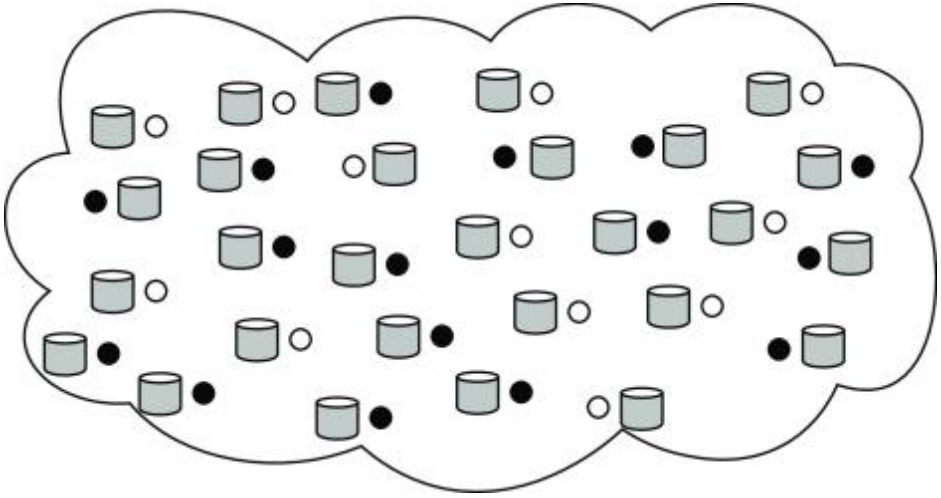
) يمضي ويلحق الكتلة.

.Block



معا إلى توافق

موثوقا، طالما أن



(leader election).

كتلة من المعاملات المتحقق منها.

بشكل عادل وقوي وآمن

(Distributed Syst

(01

(Processes)

(Introduction to T

ت الفيزيائية:

يذها Processor:

(Context

- Processor c
ل: stack pointer،
add
- Thread context: رة

ن. يمكن إجراء تبديل
عن نظام التشغيل.

لأنه يتضمن

(Thre؟

(Avoid needless): في
د تنفيذ I/O؛ أما في
ام التشغيل تحويل
ة.